WHITE ALUBMN
c
指针体系（一级/二级指针、指针与数组、指针与
字符串）
指针基础
什么是指针？ 指向另外一种类型的复合类型。
指针的大小：在 64 位计算机中，指针占 8 个字
节空间。
指向常量对象的指针：常量指针
指向函数的指针：函数指针
指向对象成员的指针，包括指向对象成员函数的
指针和指向对象成员变量的指针。特别注意：定
义指向成员函数的指针时，要标明指针所属的类.
this 指针：指向类的当前对象的指针常量。
什么是野指针和悬空指针？
悬空指针：若指针指向一块内存空间，当这块内
存空间被释放后，该指针依然指向这块a空间，
此时，称该指针为“悬空指针”。
“野指针”是指不确定其指向的指针，未初始化的
指针为“野指针”
指针和引用的区别是什么？
指针所指向的内存空间在程序运行过程中可以改
变，而引用所绑定的对象一旦绑定就不能改变。
（是否可变）
指针本身在内存中占有内存空间，引用相当于变
量的别名，在内存中不占内存空间。（是否占内
存）
指针可以为空，但是引用必须绑定对象。（是否
可为空）
指针可以有多级，但是引用只能一级。（是否能
为多级）
常量指针和指针常量的区别是什么？
常量指针：常量指针本质上是个指针，只不过这
个指针指向的对象是常量。
特点：const 的位置在指针声明运算符 *的左侧。
只要 const 位于* 的左侧，⽆论它在类型名的左
边或右边，都表⽰指向常量的指针。（可以这样
理解，* 左侧表⽰指针指向的对象，该对象为常
量，那么该指针为常量指针。）
注意 1：指针指向的对象不能通过这个指针来修
改，也就是说常量指针可以被赋值为变量的地
址，之所以叫做常量指针，是限制了通过这个指
针修改变量的值。
注意 2：虽然常量指针指向的对象不能变化，可
是因为常量指针本⾝是⼀个变量，因此，可以被
重新赋值。
指针常量的本质上是个常量，只不过这个常量的
值是⼀个指针，和上⾯那个恰恰相反，不可以修
改指针的指向，但可以通过对指针解引⽤修改指
针指向的内容。
特点：const 位于指针声明操作符右侧，表明该
对象本⾝是⼀个常量，左侧表⽰该指针指向的类
型，即以为分界线，其左侧表⽰指针指向的类
型，右侧表⽰指针本⾝的性质。
注意 1：指针常量的值是指针，这个值因为是常
量，所以指针本⾝不能改变。
注意 2：指针的内容可以改变。
函数指针和指针函数的区别是什么？
指针函数本质是⼀个函数，只不过该函数的返回
值是⼀个指针。
函数指针本质是⼀个指针变量，只不过这个指针
指向⼀个函数。
两者区别
函数指针是个指针，专门指向函数的指针变量，
⽐如能存某个函数的地址，通过它能调⽤对应的
函数。
指针函数则是个函数，只是这个函数的返回值是
指针类型，调⽤后会得到⼀个指针结果。
指针和引⽤值传递的区别是什么？
值传递是指将参数的值复制⼀份，传递给函数或
⽅法进⾏操作。在值传递中，函数或⽅法对参数
进⾏修改不会影响到原始的变量值。
指针引⽤是指将参数的内存地址传递给函数或⽅
法，使得函数或⽅法可以直接访问和修改原始变
量的值。在指针引⽤中，函数或⽅法对参数的修
改会直接反映在原始变量上。
一级指针和二级指针的区别
一级指针：存储普通变量的地址，通过 `*` 解引用
访问变量的值。
二级指针：存储一级指针的地址，通过 `**` 两次
解引用访问最终变量的值。常用于函数中修改指
针本身的值（如动态内存分配、链表节点操
作）。
指针与数组
指针与数组的区别？
数组
本质一组连续内存空间的集合
赋值数组名是常量，不能被赋值
sizeof 返回整个数组大小
传参退化为指针，丢失长度信息
指针
本质存储地址的变量
赋值指针变量可以重新指向
sizeof 返回指针本身大小（4/8字节）
传参本身就是指针
指针数组和数组指针的区别？
指针数组
数组指针
指针与字符串指针与字符串的关系？
字符串在 C 语言中以 \0 结尾的字符数组表示。
指针可以指向字符串常量或字符数组。
掌握内存管理
栈和堆的区别？
malloc 和 free 的使用注意事项？ 核心原则：谁分配，谁释放；分配后检查
NULL；释放后置 NULL。
常见内存错误及检测工具
内存对齐是什么？为什么需要内存对齐？
内存对齐规则
1. 每个成员变量的起始地址必须是其大小的整数
倍
2. 结构体总大小必须是最大成员大小的整数倍
3. 可以使用 #pragma pack(n) 修改对齐方式
为什么需要对齐
性能：CPU 按字长读取内存，对齐后一次读取即
可，不对齐需要多次读取
硬件限制：某些架构（如 ARM）要求严格对齐，
否则触发异常
结构体和共用体
结构体和共用体的区别？
联合体和结构体都是由若⼲个数据类型不同的数
据成员组成。使⽤时，联合体只有⼀个有效的成
员；⽽结构体所有的成员都有效。
对联合体的不同成员赋值，将会对覆盖其他成员
的值，⽽对于结构体的对不同成员赋值时，相互
不影响。
联合体的⼤⼩为其内部所有变量的最⼤值，按照
最⼤类型的倍数进⾏分配⼤⼩；结构体分配内存
的⼤⼩遵循内存对齐原则。
C 和 C++ struct 的区别？
在 C 语⾔中 struct 是⽤户⾃定义数据类型；在
C++ 中 struct 是抽象数据类型，⽀持成员函数的
定义。
C 语⾔中 struct 没有访问权限的设置，是⼀些变
量的集合体，不能定义成员函数；C++ 中 struct
可以和类⼀样，有访问权限，并可以定义成员函
数。
C 语⾔中 struct 定义的⾃定义数据类型，在定义
该类型的变量时，需要加上 struct 关键字，例
如：struct A var;，定义 A 类型的变量；⽽ C++
中，不⽤加该关键字，例如：A var;
C++ 中 struct和Class区别是什么？
默认访问权限不同：struct 默认访问权限为
public，class 默认访问权限为 private。
默认继承⽅式不同：当使⽤继承时，两者的默认
继承权限也不同，struct 默认继承⽅式为 public
继承，class 默认继承⽅式为 private 继承。
枚举的作用和使用场景
枚举用于定义一组命名的整数常量，提高代码可
读性
枚举与宏定义的对比
宏定义和 typedef 的区别
原理：#define 作为预处理指令，在编译预处理
时进⾏替换操作，不作正确性检查，只有在编译
已被展开的源程序时才会发现可能的错误并报
错。typedef 是关键字，在编译时处理，有类型
检查功能，⽤来给⼀个已经存在的类型⼀个别
名，但不能在⼀个函数定义⾥⾯使⽤ typedef 。
功能：typedef ⽤来定义类型的别名，⽅便使
⽤。#define 不仅可以为类型取别名，还可以定
义常量、变量、编译开关等。
作⽤域：#define 没有作⽤域的限制，只要是之
前预定义过的宏，在以后的程序中都可以使⽤，
⽽ typedef 有⾃⼰的作⽤域。
指针的操作：typedef 和 #define 在处理指针时
不完全⼀样
文件 I/O 的基本操作？
文件打开模式
预处理指令有哪些？
宏定义
条件编译
文件包含
其他预处理指令
介绍const 作⽤及⽤法
作⽤
const 修饰成员变量，定义成 const 常量，相较
于宏常量，可进⾏类型检查，节省内存空间，提
⾼了效率。
const 修饰函数参数，使得传递过来的函数参数
的值不能改变
const 修饰成员函数，使得成员函数不能修改任
何类型的成员变量（mutable 修饰的变量除
外），也不能调⽤⾮ const 成员函数，因为⾮
const 成员函数可能会修改成员变量。
类中的⽤法
const 成员变量
const 成员变量只能在类内声明、定义，在构造
函数初始化列表中初始化。
const 成员变量只在某个对象的⽣存周期内是常
量，对于整个类⽽⾔却是可变的，因为类可以创
建多个对象，不同类的 const 成员变量的值是不
同的。因此不能在类的声明中初始化 const 成员
变量，类的对象还没有创建，编译器不知道他的
值
const 成员函数
不能修改成员变量的值，除⾮有 mutable 修饰；
只能访问成员变量。
不能调⽤⾮常量成员函数，以防修改成员变量的
值
define 和 const 的区别是什么
编译阶段：define 是在编译预处理阶段进⾏替
换，const 是在编译阶段确定其值。
安全性：define 定义的宏常量没有数据类型，只
是进⾏简单的替换，不会进⾏类型安全的检查；
const 定义的常量是有类型的，是要进⾏判断
的，可以避免⼀些低级的错误。
内存占⽤：define 定义的宏常量，在程序中使⽤
多少次就会进⾏多少次替换，内存中有多个备
份，占⽤的是代码段的空间；const 定义的常量
占⽤静态存储区的空间，程序运⾏过程中只有⼀
份。
调试：define 定义的宏常量不能调试，因为在预
编译阶段就已经进⾏替换了；cons定义的常量可
以进⾏调
试。
const 的优点
有数据类型，在定义式可进⾏安全性检查
可调试
占⽤较少的空间
大端和小端的区别？
大端：高字节存储在低地址
小端：低字节存储在低地址
c++
C++ static作⽤是什么
static 定义静态变量，静态函数.保持变量内容持
久：static 作⽤于局部变量，改变了局部变量的
⽣存周期，使得该变量存在于定义后直到程序运
⾏结束的这段时间。
隐藏：static作⽤于全局变量和函数，改变了全
局变量和函数的作⽤域，使得全局变量和函数只
能在定义它的⽂件中使⽤，在源⽂件中不具有全
局可见性。（注：普通全局变量和函数具有全局
可见性，即其他的源⽂件也可以使⽤。）
static 作⽤于类的成员变量和类的成员函数，使
得类变量或者类成员函数和类有关，也就是说可
以不定义类的对象就可以通过类访问这些静态成
员。注意：类的静态成员函数中只能访问静态成
员变量或者静态成员函数，不能将静态成员函数
定义成虚函数。
static 在类中使⽤的注意事项有哪些
静态成员变量是在类内进⾏声明，在类外进⾏定
义和初始化，在类外进⾏定义和初始化的时候不
要出现static关键字和private、public、
protected 访问规则。
静态成员变量相当于类域中的全局变量，被类的
所有对象所共享，包括派⽣类的对象。
静态成员变量可以作为成员函数的参数，⽽普通
成员变量不可以。
静态数据成员的类型可以是所属类的类型，⽽普
通数据成员的类型只能是该类类型的指针或引
⽤。
static 静态成员函数
静态成员函数不能调⽤⾮静态成员变量或者⾮静
态成员函数，因为静态成员函数没有 this 指针。
静态成员函数做为类作⽤域的全局函数。
静态成员函数不能声明成虚函数（virtual）、
const 函数和 volatile 函数。
static 全局变量和普通全局变量的异同是什么
相同点存储⽅式：普通全局变量和 static 全局变量都是
静态存储⽅式。
不同点
作⽤域：普通全局变量的作⽤域是整个源程序，
当⼀个源程序由多个源⽂件组成时，普通全局变
量在各个源⽂件中都是有效的；静态全局变量则
限制了其作⽤域，即只在定义该变量的源⽂件内
有效，在同⼀源程序的其它源⽂件中不能使⽤
它。由于静态全局变量的作⽤域限于⼀个源⽂件
内，只能为该源⽂件内的函数公⽤，因此可以避
免在其他源⽂件中引起错误。
初始化：静态全局变量只初始化⼀次，防⽌在其
他⽂件中使⽤。
C++ 静态变量的使⽤场景是什么？未初始化的全
局静态变量呢？
全局静态变量（static 修饰的全局变量）
作⽤：限制变量仅在当前⽂件内可见（避免不同
⽂件中同名变量冲突），但⽣命周期是整个程序
运⾏期间。
场景：当多个⽂件需要独⽴使⽤同名变量（如统
计各模块的内部计数），但不希望被其他⽂件访
问或修改时。例：static int count = 0;（仅当前
.cpp ⽂件可访问，其他⽂件即使声明 extern int
count 也⽆法使⽤）。
局部静态变量（函数内的 static 变量）
作⽤：变量在函数第⼀次调⽤时初始化，后续调
⽤不再重新初始化，值会被保留（⽣命周期全
局，作⽤域仅限函数内）。
场景：记录函数被调⽤的次数（如 static int
call_count = 0; call_count++;）；单例模式中，
确保全局只存在⼀个实例（如函数内返回静态对
象的指针）；避免频繁创建销毁临时对象（如⼯
具函数中复⽤的缓冲区）。
类静态成员变量（static 修饰的类成员）
作⽤：属于整个类⽽⾮某个对象，所有对象共享
该变量，⽣命周期全局，需在类外单独初始化。
场景：统计类的实例数量（如 static int total;，
在构造函数中 total++，析构函数中 total--）；
存储类级别的常量或共享配置（如 static const
int MAX_SIZE = 100;）。
未初始化的全局静态变量
⾃动初始化：编译器会将其默认初始化为 0（包
括数值类型为 0，指针类型为 nullptr 等）。这是
因为全局静态变量存放在内存的 BSS 段（未初始
化数据段），程序启动时系统会⾃动将该段所有
数据清零。
作⽤域限制：和初始化的全局静态变量⼀样，仅
在当前⽂件内可见，不影响其他⽂件的同名变
量。
深入理解面向对象三大特性（封装、继承、多
态）
什么是面向对象？面向对象的三大特性
⾯向对象：对象是指具体的某⼀个事物，这些事
物的抽象就是类，类中包含数据（成员变量）和
动作（成员⽅法）。
面向对象的三大特性
封装：将具体的实现过程和数据封装成一个函
数，只能通过接口进行访问，降低耦合性。
继承：⼦类继承⽗类的特征和⾏为，⼦类有⽗类
的⾮ private ⽅法或成员变量，⼦类可以对⽗类
的⽅法进⾏重写，增强了类之间的耦合性，但是
当⽗类中的成员变量、成员函数或者类本⾝被
final 关键字修饰时，修饰的类不能继承，修饰的
成员不能重写或修改。
多态：多态就是不同继承类的对象，对同⼀消息
做出不同的响应，基类的指针指向或绑定到派⽣
类的对象，使得基类指针呈现不同的表现⽅式。
C++特性介绍⼀下？
封装是将⼀些数据和函数封装到类中，这样外层
调⽤类只会调⽤到设计者想让他调⽤的⽅法
继承的话，我常是设计⼀个基类，然后分别设置
⼦类去继承基类的⼀些⽅法，尤其是虚函数，针
对不同⼦类的特点对虚函数进⾏重写。
继承还有公有和私有两种⽅法，公有继承是将基
类的成员都原封不动的继承下来，私有继承则会
将其改为私有部分；多态的话，是有函数重载和
之前提到的虚函数，函数重载是可以使得相同的
函数⾯对不同的参数个数或者类型进⾏不同的⽅
式实现。
如何理解 C++ 是⾯向对象编程
⾯向过程编程：⼀种以执⾏程序操作的过程或函
数为中⼼编写软件的⽅法。程序的数据通常存储
在变量中，与这些过程是分开的。所以必须将变
量传递给需要使⽤它们的函数。缺点：随着程序
变得越来越复杂，程序数据与运⾏代码的分离可
能会导致问题。例如，程序的规范经常会发⽣变
化，从⽽需要更改数据的格式或数据结构的设
计。当数据结构发⽣变化时，对数据进⾏操作的
代码也必须更改为接受新的格式。查找需要更改
的所有代码会为程序员带来额外的⼯作，并增加
了使代码出现错误的机会。
⾯向对象编程（Object-Oriented Programming,
OOP）：以创建和使⽤对象为中⼼。⼀个对象
（Object）就是⼀个软件实体，它将数据和程序
在⼀个单元中组合起来。对象的数据项，也称为
其属性，存储在成员变量中。对象执⾏的过程被
称为其成员函数。将对象的数据和过程绑定在⼀
起则被称为封装。
(结合我自身的项目去说明)⾯向对象编程将数据
成员和成员函数封装到⼀个类中，并声明数据成
员和成员函数的访问级别（public、private、
protected），以便控制类对象对数据成员和函
数的访问，对数据成员起到⼀定的保护作⽤。⽽
且在类的对象调⽤成员函数时，只需知道成员函
数的名、参数列表以及返回值类型即可，⽆需了
解其函数的实现原理。当类内部的数据成员或者
成员函数发⽣改变时，不影响类外部的代码。
虚函数与虚函数表原理
C++的多态是什么？怎么通过虚函数实现
C++中的多态性是指的是同⼀个操作作⽤于不同
的对象时，可以产⽣不同的⾏为。多态性主要通
过虚函数实现，能够让你以⽗类的指针或引⽤调
⽤⼦类的实现，从⽽在运⾏时决定使⽤哪个函
数。
C++中的多态通常分为两种主要类型
编译时多态（静态多态）：通过函数重载和运算
符重载实现，在编译时确定调⽤哪个函数
运⾏时多态（动态多态）：通过虚函数实现，在
运⾏时根据对象的实际类型决定调⽤的函数。实
现原理是，每个包含虚函数的类都有⼀个虚函数
表，这个表记录了该类对象可以调⽤的虚函数指
针，每个对象也有⼀个隐式的虚函数指针，指向
其类的虚函数表。在运⾏时，调⽤虚函数时，程
序通过虚函数指针查找正确的函数。
虚函数是通过在基类中声明⼀个函数为virtual来
实现的。这标志着这个函数可以被派⽣类重写
（override）。当通过基类指针或引⽤调⽤虚函
数时，C++会查找实际对象的类型，调⽤对应的
⼦类实现，⽽不是基类的实现。
C++中的多态怎么实现的
编译时多态：也称为静态多态或早绑定。这种多
态是通过函数重载和模板来实现的。
运⾏时多态：也称为动态多态或晚绑定。这种多
态是通过虚函数和继承来实现的。当基类的指针
或引⽤指向派⽣类对象时，调⽤的虚函数将是派
⽣类的版本，这就实现了运⾏时多态。
C++的函数对象是什么？跟普通函数的区别？
函数对象是指⼀个重载了 operator() 的类或结构
体实例。函数对象可以像普通函数⼀样被调⽤，
但它们实际上是对象，具有状态和⾏为。
普通函数与函数对象的区别
定时⽅式：普通函数是⽤ 返回类型函数名(参数)
语法定义的，⽽函数对象是⼀个类或结构体，并
重载了operator()。
状态：普通函数⽆状态，⽽函数对象可以有内置
的状态（成员变量）。
调⽤⽅式：普通函数被直接调⽤，函数对象需要
先创建实例，然后⽤实例调⽤。
灵活性：函数对象可以重载多个操作符或添加更
多功能，普通函数则只能定义⼀个函数。
什么是纯虚函数？有哪些应⽤场景
纯虚函数是在基类中声明的虚函数，它在基类中
没有定义，但要求任何派⽣类都要定义⾃⼰的实
现⽅法。
含有纯虚函数的类被称为抽象类。抽象类不能被
实例化，只能作为接口使⽤。派⽣类必须实现所
有的纯虚函数，否则该派⽣类也会变成抽象类。
应用场景
设计模式：例如在模板⽅法模式中，基类定义⼀
个算法的⾻架，⽽将⼀些步骤延迟到⼦类中。这
些需要在⼦类中实现的步骤就可以声明为纯虚函
数。
接⼜定义：可以创建⼀个只包含纯虚函数的抽象
类作为接⼜。所有实现该接⼜的类都必须提供这
些函数的实现
虚函数和纯虚函数的区别？
虚函数和纯虚函数可以出现在同⼀个类中，该类
称为抽象基类。（含有纯虚函数的类称为抽象基
类）
使⽤⽅式不同：虚函数可以直接使⽤，纯虚函数
必须在派⽣类中实现后才能使⽤；
定义形式不同：虚函数在定义时在普通函数的基
础上加上 virtual 关键字，纯虚函数定义时除了加
上virtual 关键字还需要加上 =0;
虚函数必须实现，否则编译器会报错
对于实现纯虚函数的派⽣类，该纯虚函数在派⽣
类中被称为虚函数，虚函数和纯虚函数都可以在
派⽣类中重写；
析构函数最好定义为虚函数，特别是对于含有继
承关系的类；析构函数可以定义为纯虚函数，此
时，其所在的类为抽象基类，不能创建实例化对
象。
虚函数的实现机制是什么
实现机制：虚函数通过虚函数表来实现。虚函数
的地址保存在虚函数表中，在类的对象所在的内
存空间中，保存了指向虚函数表的指针（称为“虚
表指针”），通过虚表指针可以找到类对应的虚函
数表。虚函数表解决了基类和派⽣类的继承问题
和类中成员函数的覆盖问题，当⽤基类的指针来
操作⼀个派⽣类的时候，这张虚函数表就指明了
实际应该调⽤的函数
虚函数表相关知识点
虚函数表存放的内容：类的虚函数的地址
虚函数表建⽴的时间：编译阶段，即程序的编译
过程中会将虚函数的地址放在虚函数表中。
虚表指针保存的位置：虚表指针存放在对象的内
存空间中最前⾯的位置，这是为了保证正确取到
虚函数的偏移量。
虚函数表和类绑定，虚表指针和对象绑定。即类
的不同的对象的虚函数表是⼀样的，但是每个对
象都有⾃⼰的虚表指针，来指向类的虚函数表。
单继承和多继承的虚函数表结构是怎样的
编译器将虚函数表的指针放在类的实例对象的内
存空间中，该对象调⽤该类的虚函数时，通过指
针找到虚函数表，根据虚函数表中存放的虚函数
的地址找到对应的虚函数。
如果派⽣类没有重新定义基类的虚函数 A，则派
⽣类的虚函数表中保存的是基类的虚函数 A 的地
址，也就是说基类和派⽣类的虚函数 A 的地址是
⼀样的。
如果派⽣类重写了基类的某个虚函数 B，则派⽣
的虚函数表中保存的是重写后的虚函数 B 的地
址，也就是说虚函数 B 有两个版本，分别存放在
基类和派⽣类的虚函数表中。
如果派⽣类重新定义了新的虚函数 C，派⽣类的
虚函数表保存新的虚函数 C 的地址。
C++11 新特性
左值和右值的区别？左值引⽤和右值引⽤的区
别，如何将左值转换成右值？
左值：指表达式结束后依然存在的持久对象
右值：表达式结束就不再存在的临时对象。
左值和右值的区别：左值持久，右值短暂
右值引⽤和左值引⽤的区别
左值引⽤不能绑定到要转换的表达式、字⾯常量
或返回右值的表达式。右值引⽤恰好相反，可以
绑定到这类表达式，但不能绑定到⼀个左值上。
右值引⽤必须绑定到右值的引⽤，通过 && 获
得。右值引⽤只能绑定到⼀个将要销毁的对象
上，因此可以⾃由地移动其资源。
std::move 可以将⼀个左值强制转化为右值，继⽽
可以通过右值引⽤使⽤该值，以⽤于移动语义。
右值引⽤有什么作⽤？ 右值引⽤的作⽤主要有两个
可以通过右值引⽤来实现移动语义。移动语义可
以在不进⾏深拷贝的情况下，将对象的资源所有
权从⼀个对象转移到另⼀个对象，从⽽提⾼代码
的效率。
右值引⽤还可以⽤于完美转发。在函数模板中，
通过使⽤右值引⽤类型的形参来接收参数，可以
实现完美转发，即保持原参数的值类别（左值还
是右值），将参数传递给另⼀个函数。
介绍移动语义
移动语义通过右值引⽤（Rvalue References）和
移动构造函数（Move Constructors）来优化对
象在内存中的传递和处理，避免不必要的数据复
制。
介绍⼀下智能指针
在C++中，有三种常⽤的智能指针：
std::unique_ptr、std::shared_ptr和
std::weak_ptr。
std::unique_ptr：std::unique_ptr是⼀种独占所有
权的智能指针。它通过使⽤独占所有权的⽅式来
管理资
源，只能有⼀个std::unique_ptr指向同⼀个对象
或数组。当std::unique_ptr超出作⽤域或被显式
释放时，它
会⾃动删除所管理的对象或数组。它通常⽤于表
⽰独占的资源所有权，如动态分配的单个对象或
数组。
std::shared_ptr：std::shared_ptr是⼀种共享所有
权的智能指针。它可以有多个std::shared_ptr指
向同⼀个对
象，通过引⽤计数来管理资源的⽣命周期。只有
当最后⼀个std::shared_ptr超出作⽤域或被显式
释放时，资
源才会被删除。std::shared_ptr允许多个指针共
享对同⼀资源的访问，通常⽤于表⽰共享的资源
所有权
std::weak_ptr：std::weak_ptr是⼀种弱引⽤的智
能指针。它可以指向由std::shared_ptr管理的对
象，但不会
增加引⽤计数。std::weak_ptr主要⽤于解决
std::shared_ptr的循环引⽤问题，通过
std::weak_ptr.lock()⽅法
可以获取⼀个有效的std::shared_ptr来访问被管
理的对象。
在哪些场景下会应⽤智能指针我⾃⼰是在在动态内存管理中，使⽤智能指针可
以避免⼿动管理内存的⿇烦和出错风险。
shared_ptr的作⽤是什么
在传统的 C++ 编程中，使⽤ new 操作符分配的
内存需要⼿动使⽤ delete 操作符释放，若忘记释
放或者在异常情况下⽆法执⾏ delete 操作，就会
造成内存泄漏。
std::shared_ptr 可以⾃动处理内存的释放，当不
再有 std::shared_ptr 指向该对象时，对象的内存
会被⾃动释放。
std::shared_ptr ⽀持多个 std::shared_ptr 实例共
享同⼀个对象的所有权。它通过引⽤计数机制来
实现这⼀点，每个 std::shared_ptr 都会维护⼀个
引⽤计数，记录有多少个 std::shared_ptr 共享同
⼀个对象。当引⽤计数变为 0 时，对象的内存会
被释放。
weak_ptr的作⽤是什么？如何和shared_ptr结合
使⽤？
std::weak_ptr 主要⽤于辅助 std::shared_ptr 进⾏
内存管理，解决 std::shared_ptr 可能存在的循环
引⽤问题，同时还可以⽤于观察 std::shared_ptr
所管理对象的⽣命周期。
当两个或多个 std::shared_ptr 相互引⽤形成循环
时，会导致引⽤计数永远不会降为 0，从⽽造成
内存泄漏。std::weak_ptr 不会增加所指向对象的
引⽤计数，因此可以打破这种循环引⽤。
std::weak_ptr 可以⽤于观察 std::shared_ptr 所管
理对象的⽣命周期。通过 std::weak_ptr 的
expired() ⽅法可以检查所指向的对象是否已经被
释放。
⼀个 unique_ptr 怎么赋值给另⼀个 unique_ptr
对象？
借助 std::move() 可以实现将⼀个 unique_ptr 对
象赋值给另⼀个 unique_ptr 对象，其⽬的是实现
所有权的转移。
也可以让当前unique_ptr调⽤release释放对ptr控
制权，然后在另⼀个unique_ptr获取控制权。
C++11 新特性了解哪些内容
类型推导与简化语法
auto 类型推导的原理是什么？
auto 类型推导的原理：编译器根据初始值来推算
变量的类型，要求⽤ auto 定义变量时必须有初
始值。编译器推断出来的 auto 类型有时和初始
值类型并不完全⼀样，编译器会适当改变结果类
型使其更符合初始化规则。
右值引⽤与移动语义
智能指针
函数与模板增强
lambda表达式的原理是什么
从本质上讲，Lambda 表达式是编译器⾃动⽣成
的⼀个匿名的函数对象（也称为仿函数）。当我
们编写⼀个Lambda 表达式时，编译器会创建⼀
个未命名的类，这个类重载了函数调⽤运算符
operator()。
lambda 表达式的语法
语法： [捕获列表](参数列表) mutable -> 返回类
型 { 函数体 }
并发编程⽀持
C++ 11 nullptr ⽐ NULL 优势是什么
两者的区别
NULL：预处理变量，是⼀个宏，它的值是 0，定
义在头⽂件中，即 #define NULL 0
nullptr：C++ 11 中的关键字，是⼀种特殊类型的
字⾯值，可以被转换成任意其他类型。
nullptr 的优势
有类型，类型是 typdef decltype(nullptr)
nullptr_t;，使⽤ nullptr 提⾼代码的健壮性。
函数重载：因为 NULL 本质上是 0，在函数调⽤
过程中，若出现函数重载并且传递的实参是
NULL，可能会出现，不知和哪⼀个函数匹配的情
况；但是传递实参 nullptr 就不会出现这种情
况。
C++ STL
序列式容器：按元素插⼊顺序存储，元素位置与
插⼊顺序相关，不⾃动排序。
vector：动态数组，内存连续，⽀持随机访问
（[] 或 at()），优点尾插 / 尾删效率⾼（O
(1)），适合频繁访问元素的场景。缺点是中间插
⼊ / 删除效率低（O (n)），扩容时可能重新分配
内存。
deque：双端队列，内存分段连续，⽀持⾸尾⾼
效操作。优点头插 / 头删、尾插 / 尾删效率均为
O (1)，可随机访问，适合场景需要在两端频繁操
作的场景（如实现队列、栈）。
list：双向链表，元素通过指针连接，内存不连
续。优点任意位置插⼊ / 删除效率⾼（O (1)，只
需修改指针）。缺点不⽀持随机访问（访问元素
需遍历，O (n)），内存开销较⼤。
array：固定⼤⼩数组（C++11 新增），编译时确
定⼤⼩，内存连续，⽐原⽣数组更安全（⽀持边
界检查），但⼤⼩不可变。
有序关联容器（基于红⿊树实现）
set：存储唯⼀键值，元素⾃动按键升序排序，键
即值（key=value）。适合场景需要去重且有序的
数据集合（如存储不重复的 ID 并排序）。
multiset：与 set 类似，但允许键值重复，其他特
性相同。
map：存储键值对（key-value），键唯⼀且⾃动
排序，通过键快速查找值。适合场景键值映射场
景（如字典、配置表）。
multimap：与 map 类似，但允许键重复（⼀个
键可对应多个值）。
⽆序关联容器（C++11 新增，基于哈希表实现）
unordered_set / unordered_multiset：功能同
set / multiset，但元素⽆序，通过哈希表实现，
查找、插⼊、删除平均效率更⾼（O (1)）。适合
场景对顺序⽆要求，但需要快速增删查的场景。
unordered_map / unordered_multimap：功能同
map / multimap，⽆序，基于哈希表，平均操作
效率 O (1)。。
容器适配器：基于其他容器实现，封装特定接
⼜，提供受限功能
stack：栈，遵循 “后进先出（LIFO）”，仅⽀持在
顶部插⼊ / 删除 / 访问元素，默认基于 deque 实
现，也可指定 vector 或 list 作为底层容器。
queue：队列，遵循 “先进先出（FIFO）”，仅⽀
持在尾部插⼊、头部删除。默认基于 deque 实
现，也可指定 list 作为底层容器。
priority_queue：**优先队列，元素按优先级⾃动
排序（默认最⼤元素在顶部），插⼊ / 删除效率
O (log n)。默认基于 vector 实现，底层⽤堆结构
维护优先级。
vector中push_back和emplace_back的区别？
push_back() 向容器尾部添加元素时，⾸先会创
建这个元素，然后再将这个元素拷贝或者移动到
容器中（如果是拷贝的话，事后会⾃⾏销毁先前
创建的这个元素）；
⽽emplace_back() 在实现时，则是直接在容器尾
部创建这个元素，省去了拷贝或移动元素的过
程。
使⽤迭代器怎么删除⼀个元素
顺序容器（序列式容器，⽐如vector、deque）
删除元素的⽅式：erase迭代器不仅使所指向被删
除的迭代器失效，⽽且使被删元素之后的所有迭
代器失效(list除外)，所以不能使⽤erase(it++)的
⽅式，但是erase的返回值是下⼀个有
效迭代器：It = c.erase(it);
关联容器(关联式容器，⽐如map、set、
multimap、multiset等) 删除元素的⽅式：erase
迭代器只是被删除元素的迭
代器失效，但是返回值是void，所以要采⽤
erase(it++)的⽅式删除迭代器；c.erase(it++)
C++内存管理
C++中的内存分区有哪些
栈区（Stack）：由编译器⾃动分配释放，存放
函数的参数值，局部变量等。其操作⽅式类似于
数据结构中的栈。
堆区（Heap）：⼀般由程序员分配释放，若程序
员不释放，程序结束时可能由OS回收。注意，与
数据结构中的堆是两回事，分配⽅式倒是类似于
链表。
全局区（静态区）（Static）：全局变量和静态
变量被分配到同⼀块内存中。在C++中，全局区
还包含了常量区，字符串常量和其他常量也是存
储在此。
常量区：是全局区的⼀部分，存放常量，不允许
修改。
代码区（Text）：存放函数体的⼆进制代码。
C++中堆和栈的区别
申请⽅式
栈：由系统⾃动分配。例如，声明在函数中⼀个
局部变量 int b，系统⾃动在栈中为b开辟空间。
堆：需要程序员⾃⼰申请，并指明⼤⼩，在C语
⾔中通过malloc函数，如 p1 =
(char*)malloc(10);，在C++中⽤new运算符，如
p2 = new char[20]
申请后系统的响应
栈：只要栈的剩余空间⼤于所申请空间，系统将
为程序提供内存，否则将报异常提⽰栈溢出。
堆：⾸先应该知道操作系统有⼀个记录空闲内存
地址的链表，当系统收到程序的申请时，会遍历
该链表，寻找第⼀个空间⼤于所申请空间的堆结
点，然后将该结点从空闲结点链表中删除，并将
该结点的空间分配给程序，另外，对于⼤多数系
统，会在这块内存空间中的⾸地址处记录本次分
配的⼤⼩，这样，代码中的delete语句才能正确
的释放本内存空间。另外，由于找到的堆结点的
⼤⼩不⼀定正好等于申请的⼤⼩，系统会⾃动
的将多余的那部分重新放⼊空闲链表中。
申请⼤⼩的限制
栈：栈是向低地址扩展的数据结构，是⼀块连续
的内存的区域。这句话的意思是栈顶的地址和栈
的最⼤容量是系统预先规定好的，操作系统中，
栈的⼤⼩是⼏MB，如果申请的空间超过栈的剩余
空间时，将提⽰overflow。因此，能从栈获得的
空间较⼩。
堆：堆是向⾼地址扩展的数据结构，是不连续的
内存区域。这是由于系统是⽤链表来存储的空闲
内存地址的，⾃然是不连续的，⽽链表的遍历⽅
向是由低地址向⾼地址。堆的⼤⼩受限于计算机
系统中有效的虚拟内存。由此可见，堆获得的空
间⽐较灵活，也⽐较⼤。
⽣命周期
栈：栈的内存管理是⾃动的，变量的内存会在其
作⽤域结束时⾃动释放
堆：堆的内存管理需要⼿动进⾏，需要使⽤new
关键字分配内存，并使⽤delete或delete[]关键字
释放内存，否则会导致内存泄漏。
new是在内存上哪⼀块去分配的内存？
new所申请的内存区域在C++中称为⾃由存储
区。很多编译器的new/delete都是以malloc/free
为基础来实现的，所以通常都是借由堆实现来实
现⾃由存储，这时候就可以说new所申请的内存
区域在堆上。
如果new内存失败了会是怎么样
会抛出std::bad_alloc异常，如果加上
std::nothrow关键字，A* p = new (std::nothrow)
A;，new 就不会抛出异常⽽是会返回空指针。
C++ malloc和new的区别是什么
分配内存的位置：malloc是从堆上动态分配内
存，new是从⾃由存储区为对象动态分配内存。
⾃由存储区的位置取决于operator new的实现。
⾃由存储区不仅可以为堆，还可以是静态存储
区，这都看operator new在哪⾥为对象分配内
存。
返回类型安全性：malloc内存分配成功后返回
void*，然后再强制类型转换为需要的类型；new
操作符分配内存成功后返回与对象类型相匹配的
指针类型；因此new是符合类型安全的操作符。
内存分配失败返回值：malloc内存分配失败后返
回NULL。new分配内存失败则会抛异常
（bac_alloc）。
分配内存的⼤⼩的计算：使⽤new操作符申请内
存分配时⽆须指定内存块的⼤⼩，编译器会根据
类型信息⾃⾏计算，⽽malloc则需要显式地指出
所需内存的尺⼨。
是否可以被重载：opeartor new /operator
delete可以被重载。⽽malloc/free则不能重载。
new 和 malloc 如何判断是否申请到内存
malloc ：成功申请到内存，返回指向该内存的指
针；分配失败，返回 NULL 指针。
new ：内存分配成功，返回该对象类型的指针；
分配失败，抛出 bac_alloc 异常。
delete和free的区别
new/delete是C++的操作符，⽽malloc/free是C
中的函数。
new做两件事，⼀是分配内存，⼆是调⽤类的构
造函数；同样，delete会调⽤类的析构函数和释
放内存。⽽malloc和free只是分配和释放内存。
new建⽴的是⼀个对象，⽽malloc分配的是⼀块
内存；new建⽴的对象可以⽤成员函数访问，不
要直接访问它的地址空间；malloc分配的是⼀块
内存区域，⽤指针访问，可以在⾥⾯移动指针；
new出来的指针是带有类型信息的，⽽malloc返
回的是void指针。
new/delete是保留字，不需要头⽂件⽀持；
malloc/free需要头⽂件库函数⽀持。
delete 实现原理？delete 和 delete[] 的区别？
delete 的实现原理
⾸先执⾏该对象所属类的析构函数；
进⽽通过调⽤ operator delete 的标准库函数来
释放所占的内存空间。
delete 和 delete [] 的区别
delete ⽤来释放单个对象所占的空间，只会调⽤
⼀次析构函数
delete [] ⽤来释放数组空间，会对数组中的每个
成员都调⽤⼀次析构函数。
malloc 的原理？malloc 的底层实现？
malloc 的原理
当开辟的空间⼩于 128K 时，调⽤ brk() 函数，通
过移动 _enddata 来实现；
当开辟空间⼤于 128K 时，调⽤ mmap() 函数，
通过在虚拟地址空间中开辟⼀块内存空间来实现
malloc 的底层实现
brk() 函数实现原理：向⾼地址的⽅向移动指向数
据段的⾼地址的指针 _enddata。
mmap 内存映射原理：
1. 进程启动映射过程，并在虚拟地址空间中为映
射创建虚拟映射区域；
2. 调⽤内核空间的系统调⽤函数 mmap()，实现
⽂件物理地址和进程虚拟地址的⼀⼀映射关系；
3. 进程发起对这⽚映射空间的访问，引发缺页异
常，实现⽂件内容到物理内存（主存）的拷贝。
介绍⼀下malloc的brk，mmap
单例、工厂、观察者
单例模式核心思想：确保一个类只有一个实例，并提供全
局访问点。
懒汉式
饿汉式
工厂模式的实现核心思想：将对象的创建和使用分离，通过工厂
方法创建对象。
观察者模式的实现
核心思想：定义对象间的一对多依赖关系，当一
个对象状态改变时，所有依赖者自动收到通知。
Qt 开发
Qt 信号与槽机制的原理？
信号与槽是 Qt 的核心机制，用于对象间通信，
是观察者模式的 Qt 实现。
信号与槽的特点
一个信号可以连接多个槽
多个信号可以连接同一个槽
信号可以连接信号
连接可以是跨线程的（QueuedConnection）
类型安全（Qt5 语法）
连接方式
Qt 的内存管理机制？ Qt 采用对象树机制管理内存：父对象销毁时自动
销毁所有子对象。
数据结构与算法
数组和链表的区别
如何判断链表是否有环？如何找到环的入口？ 快慢指针法（Floyd 判圈算法）
栈和队列的区别及相互实现区别
栈（Stack）：后进先出（LIFO），操作在栈顶
队列（Queue）：先进先出（FIFO），操作在队
头和队尾。
最小栈如何实现？
设计一个栈，支持 push、pop、top，并在 O(1)
时间内检索到最小元素。
哈希表的底层原理和冲突解决？
哈希表：通过哈希函数将键映射到数组索引，实
现 O(1) 平均查找。
冲突解决
链地址法（Separate Chaining）：每个桶是一个
链表，冲突时追加到链表。Java HashMap、C++
unordered_map 使用此方法。
开放地址法（Open Addressing）：冲突时按探
测序列找下一个空位
线性探测： h(k) + i
二次探测： h(k) + i²
双重哈希： h1(k) + i * h2(k)
C++ unordered_map 底层
底层是哈希桶数组，每个桶是链表（链地址
法）。
负载因子（load_factor）= 元素数 / 桶数。当负
载因子超过阈值（通常 1.0）时，触发 rehash
（扩容并重新散列）。
如何判断二叉树是否平衡？
平衡二叉树（AVL）：任意节点的左右子树高度
差不超过 1
图的存储方式？
邻接矩阵：二维数组 adj[i][j] 表示边。适合稠密
图，空间 O(V²)。
邻接表：数组 + 链表（或 vector）。适合稀疏
图，空间 O(V + E)。
图的遍历（BFS 和 DFS）
BFS（广度优先）：用队列，适合求最短路径
（无权图）
DFS（深度优先）：用递归或栈，适合拓扑排
序、连通分量。
最短路径算法有哪些
常见排序算法的复杂度对比
快速排序的实现和原理
原理：分治法。选一个基准（pivot），将数组分
为小于和大于基准的两部分，递归排序。
归并排序的实现？
原理：分治法。将数组分成两半，分别排序，然
后合并。
堆排序的实现？
原理：利用大顶堆（或小顶堆）的性质。先建
堆，然后依次将堆顶元素与末尾交换，调整堆。
动态规划的核心思想？
核心：将大问题分解为重叠子问题，通过保存子
问题的解（备忘录/DP 表）避免重复计算。
步骤
1. 定义状态（dp 数组的含义）
2. 确定状态转移方程
3. 确定初始条件（base case）
4. 确定遍历顺序
0-1 背包问题？
最长递增子序列（LIS）
分支主题 4